Artículo

Aprendizaje supervisado, clustering difuso e interpretabilidad con un enfoque KDD-XAI para la caracterización del cibertráfico IoT en un entorno controlado de forma privada

Martín José García Navarro

[1] Afiliación 1: Universidad [UEM], Madrid, España; mjpj0609@gmail.com
* Correspondencia: martinjosetrabajo@gmail.com

Resumen

En la detección de ciberataques, muchos enfoques se apoyan en características directas del flujo de paquetes, como el protocolo o servicio utilizado, los puertos y determinadas firmas conocidas. Sin embargo, estas aproximaciones pueden dejar en segundo plano relaciones menos evidentes entre variables estadísticas del tráfico.

Este estudio propone un enfoque integrado orientado a analizar patrones menos visibles a simple vista. Para ello, se realizó un análisis exploratorio de datos (EDA) con el objetivo de identificar y reducir posibles atajos, redundancias y dependencias. Posteriormente, se aplicaron técnicas de clustering de grupos anónimos (FCM-KCM) y se desarrolló una clasificación supervisada mediante aprendizaje automático (ML), añadiendo técnicas de predicción e interpretabilidad (XAI).

Los resultados permitieron evaluar la utilidad, sobre todo de variables temporales, volumétricas y estructurales asociadas al comportamiento del tráfico IOT, y aportó valor al no depender únicamente de indicadores directos del entorno y no precisando de información privada dentro del flujo, sino de patrones estadísticos asociados al comportamiento del tráfico.

Keywords: ciberseguridad; IoT; RT-IoT2022; clasificación supervisada; clustering; interpretabilidad (XAI); SHAP; LIME

MSC: 62H30; 68M10; 68T05; 68T10

Martín José García Navarro

Citation: García Navarro, M. J. (2026). ***Aprendizaje supervisado, clustering difuso e interpretabilidad con un enfoque KDD-XAI para la caracterización del cibertráfico IoT en un entorno controlado.*** Zenodo. DOI: 10.5281/zenodo.20131778

Copyright © 2026 Martín José García Navarro. Este trabajo se distribuye bajo licencia Creative Commons Attribution 4.0 International (CC BY 4.0).

1. Introducción

La detección temprana de tráfico no deseado o malicioso en entornos IoT es uno de los retos más relevantes de la ciberseguridad aplicada, tanto es así que no deja de cambiar y actualizarse cada día. Esto ocurre por la heterogeneidad de los dispositivos y actualización constante de su software provocando nuevas condiciones que favorecen la aparición de tráfico no benigno.

El conjunto de datos RT-IoT2022 se diseñó precisamente para estudiar ese escenario bajo condiciones controladas, combinando tráfico benigno de dispositivos inteligentes con ataques simulados sobre una infraestructura de laboratorio [1].

El problema de investigación es doble. Por un lado, interesa determinar si la estructura latente del tráfico preserva la semántica de las etiquetas conocidas o si aparecen regiones mixtas, subgrupos o fronteras difusas entre comportamientos de forma matemática. Por otro lado, interesa cuantificar hasta qué punto simplemente los descriptores de flujo, (sin inspección del contenido de los paquetes) permiten distinguir tráfico benigno y no benigno, grupos y clases de ataque.

Este estudio presenta un framework generalizado de cuatro fases, propuesto para clasificar y controlar los patrones no evidentes dentro del tráfico sin necesidad de intrusión. Se realizará la integración mediante técnicas avanzadas de análisis (EDA) para la detección temprana de atajos conocidos, clustering difuso y no difuso, y por último modelado y aprendizaje supervisado ml, y predicción e interpretabilidad (XAI).

Las contribuciones principales del manuscrito son tres:

- Un enfoque KDD-XAI realizando un estudio EDA, la mitigación del desbalanceo y la selección de variables para evitar atajos triviales, permitiendo la selección de patrones menos evidentes.
- Conectar la fase no supervisada con la supervisada, añadiendo una herramienta para detectar regiones compactas y zonas ambiguas.
- Incorporar una arquitectura escalable y privada para instituciones y analistas de red

2. Antecedentes y trabajos relacionados

2.1 De la detección basada en firmas a la explicabilidad

La evolución histórica de los sistemas de detección de intrusiones se origina desde la creación misma de internet publicándose la primera investigación relacionada con esta área en 1967 por la Corporación RAND y la Agencia de Seguridad Nacional [2]. Estos tratan de suprimir y minimizar el riesgo de acceso indebido a información privada o bloqueada para otros usuarios mediante reglas estáticas y firmas conocidas hacia modelos capaces de aprender regularidades complejas sobre datos de tráfico. En entornos IoT, este cambio es especialmente importante por la variedad de dispositivos, protocolos y ritmos de comunicación, tranformando en ineficiencia cualquier enfoque rígido [3].

En contextos industriales, sanitarios, académicos o domésticos conectados, un sistema que genera alertas sin justificación alguna, dificulta la respuesta del analista y limita la adopción institucional. Por ello, los IDS en IoT incorpora cada vez con más frecuencia herramientas de XAI, tanto para selección de variables como para explicación local y global de predicciones [4,5]

2.2 Trabajos relacionados

Este estudio se sitúa en una línea de investigación vinculada a la detección y caracterización de tráfico IoT mediante técnicas de aprendizaje automático, clustering e

 https://doi.org/10.5281/zenodo.20125818

interpretabilidad. En los últimos años, conjuntos de datos como RT-IoT2022 han permitido analizar escenarios controlados combinando flujos de tráfico variados [1]. Dentro de este contexto, se ha mostrado que la integración de análisis exploratorio, clasificación, agrupamiento y técnicas XAI aun siendo por separado, puede mejorar tanto el rendimiento predictivo como la comprensión del comportamiento del tráfico. Además, en otros dominios científicos, trabajos recientes de Marín Díaz han mostrado que la combinación de Fuzzy C-Means, modelos predictivos e interpretabilidad permite obtener una lectura más explicativa del fenómeno estudiado sin perder capacidad de predicción [6–9]

2.3 Posicionamiento del presente trabajo

Desde el punto de vista algorítmico, el estudio combina un enfoque KDD mediante un estudio EDA, una agrupación mediante clusters difusos con Fuzzy C-Means [10] y no difusos, clasificación con XGBoost [11] y Extra Trees [12], y explicación mediante SHAP-ELI5, LIME e importancia por permutación [13–15]. El interés del manuscrito es ofrecer una visión contemporánea de la detección IDS basada en la protección y privatización de los datos.

2.4 Web of Science (WOS)

Para realizar una búsqueda intensiva y extensiva de un artículo relacionado por tema o estructura al nuestro, nos enfocamos en varias webs de investigación y publicación científica, siendo una de ellas WOS, para ello armamos un prompt que incluye el flujo de nuestro pipeline expandiendo términos para posibles candidatos. Sin embargo, la búsqueda combinada no devolvió resultados, lo que sugiere que la integración completa de estas fases -EDA, clustering clásico y difuso, clasificación e interpretabilidad XAI- no aparece de forma frecuente en los trabajos indexados recuperados mediante esta estrategia.

TS=("RT-IoT2022" OR "RT-IOT2022" OR "iot security" OR "internet of things")
AND TS=("EDA" OR "exploratory data analysis" OR "data exploration" OR "feature analysis")
AND TS=("clustering" OR "KMeans" OR "K-means" OR "fuzzy" OR "Fuzzy C-Means")
AND TS=("prediction" OR "classification" OR "machine learning")
AND TS=("SHAP" OR "LIME" OR "ELI5" OR "XAI" OR "explainable AI")

3. Materiales and Métodos

3.1 Contextualización, preparación y caracterización del dataset

3.1.1 Ecosistema controlado y disponibilidad de los datos

El dataset RT-IoT2022 esta disponible en el **UCI Machine Learning Repository** y también replicado en **Kaggle** [25]. Parte de un laboratorio con disposivos IoT benignos, y varios nodos atacantes que generan varios patrones maliciosos básicos, y por tanto el porcentaje de ataques es muchísimo mayor al porcentaje de flujo real normal. En parte esta proporción se debe a que cada cantidad de tipo de ataque principal es proporcional a los otros pero también al tráfico normal y por otra parte, se debe al sesgo de que el propio entorno ya este controlado de por sí.

 https://doi.org/10.5281/zenodo.20125818

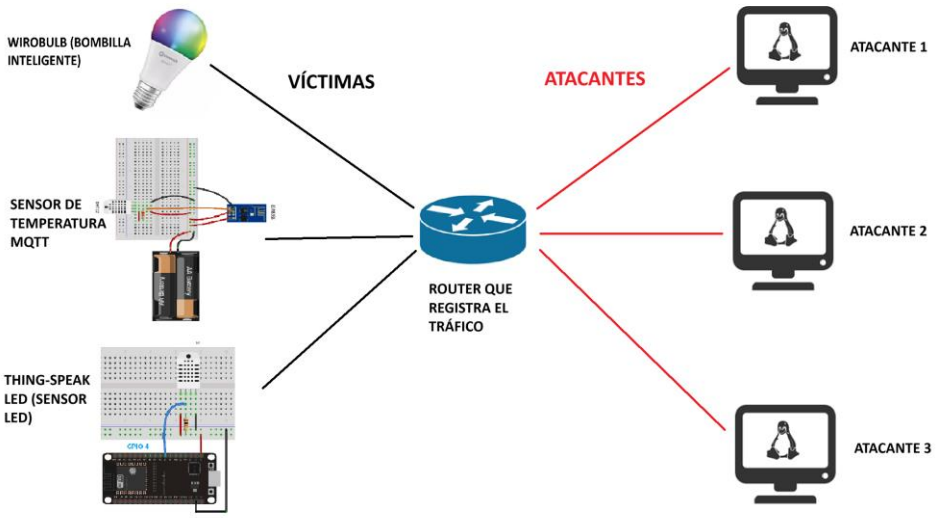


Figura 1. Escenario de laboratorio IoT utilizado para construir RT-IoT2022 sintetizando la coexistencia de tráfico benigno y nodos atacantes bajo captura controlada.

La Figura 1 resume visualmente ese entorno de captura. Las clases originales contempladas son doce, y combinan tráfico benigno, y cuatro tipos de ataques: ARP_poisioning, dos modalidades DoS/DDoS, una clase de fuerza bruta SSH y cinco variantes de exploración Nmap. Para ordenar la experimentación, el análisis utiliza tres niveles de target en las primeras fases: attack_flag (una bandera binaria) , attack_groupstr (un flag multiclase fino de los doce subgrupos desglosados) y attack_type (agregación intermedia por familia de ataque).

La Tabla A1 resume la base de trabajo

Table A1. Pipeline principal

Indicador	Valor
Observaciones iniciales del notebook	123117 flujos
Variables tras integración y renombrado	86
Observaciones tras depuración analítica	117,922
Matriz expandida usada para análisis	123 columnas
Matriz analítica final usada en ML y clustering	53 columnas
Variables efectivas sin flags para análisis	51/123
Distribución original del grupo dos_ddos	90,622 flujos (~76.85 %)
Dataset mitigado para análisis	34,375 flujos
Flujos dos_ddos retenidos en el dataset mitigado	7,075

3.1.2 Diccionario de variables y preparación del espacio analítico

Este estudio tiene más unas 123 variables que determinan estadísticas e información superficial del flujo IoT en cada observación y por tanto es necesario saber en todo momento la definición formal de estas. Se presenta un diccionario de sílabas y palabras cuya combinación puede formar una variable:

- fwd (forwards) y bwd (backwards) separan la dirección del intercambio: fwd describe el sentido origen→destino, y bwd el retorno destino→origen, ej. cliente→servidor.

- flow, down_up_ratio y flow_duration resumen el comportamiento global del flujo completo, combinando ambas direcciones, su asimetría y su duración total.

- pkts (packets), data_pkts, payload y header_size cuantifican volumen y composición del tráfico: número de paquetes, paquetes con datos útiles o vagos, bytes de carga útil y tamaño acumulado de cabeceras IP/TCP/UDP.

- iat (Intel arrival time), active e inactive capturan la dinámica temporal del flujo: ritmo entre paquetes, periodos de ráfaga y pausas o intervalos ociosos.

- subflow y bulk (block) describen estructura interna y transferencias continuas en bloque, mientras que window resume la ventana TCP empleada para el control de flujo.

- Las banderas SYN, ACK, FIN, RST, PSH, URG, ECE y CWR aportan contexto sobre apertura, confirmación, cierre, reinicio y congestión de las conexiones TCP.

- id_src_port (source port), id_dst_port (Destiny port), proto (protocol) y service añaden contexto de comunicación, puertos, protocolo y servicio detectado, aunque parte de estas señales se controla o elimina en el modelado para evitar atajos demasiado cercanos a la etiqueta.

3.2 Flujo metodológico KDD–XAI

El flujo de trabajo del pipeline se resume en la metodología KDD-XAI que consiste en un flujo continuo combinando estructuración, exploración, mitigación, agrupación, predicción y culminando en la explicación y modelado, esto se resume en la Figura 2.

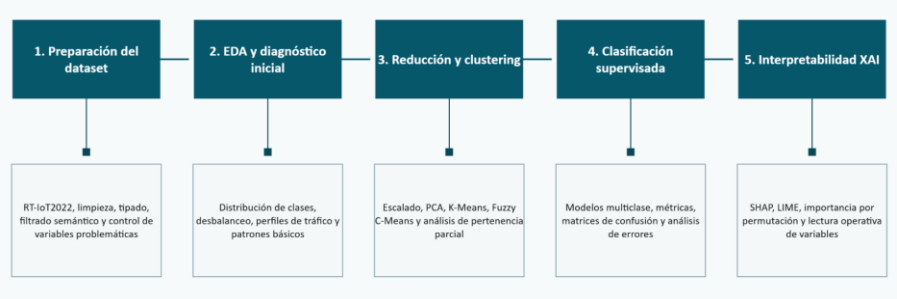


Figura 2. Esquema metodológico organizado con un flujo KDD–XAI.

3.3 Análisis no supervisado difuso y no difuso

3.3.1 Reducción de dimensionalidad y selección de grupos

 https://doi.org/10.5281/zenodo.20125818

Antes de aplicar PCA se realizó una estandarización mediante una transformación Z-score. Para cada variable x_{ij}, se restó su media μ_jy se dividió entre su desviación típica σ_j, obteniendo una matriz transformada con variables centradas en media cero y varianza en uno. Esta normalización evita que las variables con mayor escala dominen el cálculo de distancias en K-Means o la estimación de las componentes principales en PCA.

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$$

Sobre la matriz estandarizada Z, la reducción lineal por PCA se expresa como:

$$Z = U\Lambda V^T$$

Donde U Contiene las coordenadas de las observaciones en el nuevo espacio (la muestra), Λ contiene los valores asociados a la varianza explicada por cada componente principal y $\boldsymbol{V^T}$ contiene las direcciones principales transpuestas (mayor variabilidad), reuniendo las primeras (q) componentes que mayor variabilidad e información tienen respecto a la muestra, esto se hace mediante el criterio de retención:

$$\eta(q) = \frac{\sum_{j=1}^{q} \lambda_j}{\sum_{j=1}^{p} \lambda_j}$$

Donde $\eta(q)$ indica la varianza de los primeros (q) componentes de los (p) posibles.

3.3.2 Tests de selección de clusters

La elección del número de clusters K es una decisión crítica, ya que un valor demasiado bajo puede mezclar comportamientos de red diferentes dentro del mismo grupo, mientras que un valor demasiado alto puede producir sobrefragmentación y clusters difíciles de interpretar. Por este motivo, la selección de K no se apoya en una única métrica, sino en la lectura conjunta de varios criterios internos de validación.

Este estudio presenta cuatro tipos de validación de clusters:

El test de inercia, definida como la suma de distancias cuadráticas dentro de los clusters:

$$J(K) = \sum_{k=1}^{K} \sum_{z_i \in C_k} \|z_i - \mu_k\|_2^2$$

Métrica que disminuye de forma natural ya que el número de clusters (K) es inversamente proporcional a la distancia de estos y sus centroides. Aunque no siempre resulta eficiente y por ello se usa el método del codo donde un punto de inflexión dicta una gran mejoría y donde añadir más clusters no aporta nada significativo.

El coeficiente de Silhouette, mide la coherencia de cada observación con su propio cluster frente a los clusters vecinos:

 https://doi.org/10.5281/zenodo.20125818

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

donde a(i) es la distancia media de la observación i respecto a los puntos de su mismo cluster, y b(i) es la menor distancia media respecto a los puntos de otro cluster. El valor de s(i) se encuentra entre [−1,1]. Valores próximos a 1 indican una buena asignación, valores cercanos a 0 indican solapamiento entre clusters, y valores negativos sugieren que la observación podría estar mejor asignada a otro grupo.

Otra métrica considerada es el índice de Davies-Bouldin, que compara la dispersión interna de los clusters con la separación entre sus centroides:

$$DB = \frac{1}{K}\sum_{k=1}^{K} \max_{l \neq k}\left(\frac{S_k + S_l}{M_{kl}}\right)$$

donde $S_k$representa la dispersión media del cluster k, $S_l$la dispersión media de otro cluster l, y M_{kl}la distancia entre los centroides μ_ky μ_l. En este caso, valores más bajos indican una mejor estructura de agrupamiento, ya que reflejan clusters compactos y bien separados.

Finalmente, se emplea el índice de Calinski-Harabasz, que evalúa la relación entre la dispersión entre clusters y la dispersión dentro de los clusters:

$$CH = \frac{\mathrm{tr}(B_K)/(K-1)}{\mathrm{tr}(W_K)/(n-K)}$$

Donde B_K representa la matriz de dispersión entre clusters, W_K la matriz de dispersión dentro de los clusters, n el número total de observaciones y K el número de grupos. Valores altos de este índice indican una mayor separación entre grupos en relación con la variabilidad interna de cada cluster.

3.3.3 Clustering no difuso

Una de las líneas del clustering no difuso utilizadas es K-Means, donde cada observación pertenece a un único cluster, y cuyo objetivo es minimizar la suma cuadrática intracluster:

$$\min_{\{C_k\},\{\mu_k\}} \sum_{k=1}^{K} \sum_{z_i \in C_k} \|z_i - \mu_k\|_2^2$$

Donde C_k es el cluster k, z_i una observación y μ_k su centroide. Cada centroide se calcula como la media de los puntos asignados:

$$\mu_k = \frac{1}{|C_k|}\sum_{z_i \in C_k} z_i$$

Y la asignación de cada observación se realiza buscando el centroide más cercano:

 https://doi.org/10.5281/zenodo.20125818

$$c_i = \arg\min_{k\in\{1,\dots,K\}} \|z_i - \mu_k\|_2^2$$

3.3.4 Clustering difuso y ambigüedad

El modelo Fuzzy C-Means se define minimizando la función objetivo y donde cada observación se reparte en pertenencia en varios clusters al mismo:

$$J_m(U,V) = \sum_{i=1}^{n}\sum_{k=1}^{K} u_{ik}^m \, \|z_i - v_k\|_2^2$$

donde u_{ik} es el grado de pertenencia del flujo (i) al cluster k, m > 1 es el parámetro de difusidad y v_k es el centroide del cluster [10]. Las actualizaciones iterativas vienen dadas por:

$$u_{ik} = \left(\sum_{l=1}^{K}\left(\frac{\|z_i - v_k\|_2}{\|z_i - v_l\|_2}\right)^{\frac{2}{m-1}}\right)^{-1} \qquad \text{y} \qquad v_k = \frac{\sum_{i=1}^{n} u_{ik}^m z_i}{\sum_{i=1}^{n} u_{ik}^m}$$

Otras evaluaciones que existen para validar los clusters son su nitidéz, para lo que se utiliza el coeficiente de partición difusa:

$$\mathrm{FPC} = \frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} u_{ik}^2$$

que se aproxima a 1 cuando las asignaciones son muy claras.

3.4 Análisis supervisada y formulación de modelos

3.4.1 Clasificación supervisada y selección de modelos

La fase supervisada compara varios modelos con validación cruzada estratificada de 10 particiones, para elegir cual es el que mejor estima la curva que siguen los datos. Formalmente, la selección de hiperparámetros sigue el criterio:

$$\hat{\theta} = \arg\max_{\theta\in\Theta} \frac{1}{R}\sum_{r=1}^{R} \mathrm{AUC}^{(r)}(\theta)$$

En esta expresión, θ representa una configuración concreta de hiperparámetros del modelo, mientras que Θ recoge el conjunto total de configuraciones evaluadas durante la búsqueda. Para cada configuración θ, el modelo se entrena y valida mediante R particiones de validación cruzada, obteniendo un valor de $\mathrm{AUC}^{(r)}(\theta)$ en cada fold r. La suma de estos valores, dividida entre R, proporciona el AUC medio de dicha configuración y es el máximo de estos el que obtiene el máximo rendimiento.

3.4.2 Funcionamiento y formulación de los modelos

El objetivo para un modelo es contrastar familias con supuestos geométricos y estadísticos diferentes sobre el mismo espacio de variables de flujo, y comprobar cuáles capturan mejor relaciones lineales, no lineales, interacciones de alto orden y clases desbalanceadas.

 https://doi.org/10.5281/zenodo.20125818

Generalmente todos los modelos persiguen estimar una regla de decisión:

$$\hat{y}(x) = \arg\max_{c \in \mathcal{Y}} f_c(x)$$

donde $f_c(x)$ representa una puntuación, probabilidad, o función discriminante asociada a la clase c. La diferencia entre algoritmos radica en cómo se define esa función y qué hipótesis se imponen sobre los datos.

3.5 Funcionamiento de los modelos

3.5.1 XGBoost

XGBoost pertenece a la familia del gradient boosting sobre árboles, cuya formulación fue consolidada por Friedman y optimizada computacionalmente en la implementación de Chen T. y Guestrin de 2016 [11,16]. El modelo construye aditivamente un conjunto de árboles:

$$\widehat{y_i^{(T)}} = \sum_{t=1}^{T} f_t(x_i), \qquad f_t \in \mathcal{F}$$

minimizando en cada iteración una función objetivo regularizada:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} l\left(y_i, \widehat{y_i^{(t-1)}} + f_t(x_i)\right) + \Omega(f_t)$$

En términos operativos, la primera expresión indica que la predicción final no procede de un único árbol, sino de la suma acumulada de T árboles entrenados secuencialmente. Cada nuevo árbol f_t se incorpora para corregir los errores residuales del anterior $\hat{y}_i^{(t-1)}$.

La segunda expresión formaliza este proceso como una minimización iterativa: el término $l(\cdot)$ mide el error de predicción tras añadir el nuevo árbol, mientras que $\Omega(f_t)$ penaliza su complejidad estructural, equilibrando capacidad predictiva y control del sobreajuste.

3.5.2 Extratrees

El algoritmo Extremely Randomized Trees fue introducido por Geurts, Ernst y Wehenkel en 2006 [12]. Aunque comparte con Random Forest el ensamblaje de muchos árboles, incrementa la aleatoriedad al seleccionar umbrales de corte, en vez de buscar siempre el mejor umbral posible. Si una partición se define sobre la variable j, los posibles puntos de corte dentro de su rango, puede generarse aleatoriamente como:

$$\tau \sim \mathcal{U}\left(\min(x_{\cdot j}), \max(x_{\cdot j})\right)$$

para después escoger entre varios candidatos el que produzca la mejor reducción de impureza. La predicción sigue siendo:

$$\hat{y}(x) = \text{mode}\{\, h_b^{\text{ET}}(x)\}_{b=1}^{B}$$

 https://doi.org/10.5281/zenodo.20125818

Donde cada árbol h_b^{ET} emite una predicción para la observación x, y la clase asignada corresponde a la moda de las predicciones individuales. Especialmente útil en datos tabulares de tráfico IoT con múltiples variables y posibles patrones no lineales por su versatilidad, rapidez y estabilidad estructural.

3.5.3 Regresión logística

La regresión logística moderna se consolidó a mediados del siglo XX, tras la introducción del logit por Berkson y su posterior formalización en estadística aplicada [17]. En la clasificación binaria, se modela directamente la probabilidad condicional de ataque mediante:

$$\mathrm{P}(\mathrm{y}=1 \mid \mathrm{x})=\sigma(\beta_0+\beta^{\mathrm{T}}\mathrm{x})=\frac{1}{1+\mathrm{e}^{-(\beta_0+\beta^{\mathrm{T}}\mathrm{x})}}$$

En extensión multiclase, se puede enfocar como one-vs-rest o mediante softmax. Su entrenamiento minimiza la pérdida logística regularizada:

$$L(\beta)=-\sum_{i=1}^{n}[y_i\log(p_i)+(1-y_i)\log(1-p_i)]+\lambda\|\beta\|_q$$

En el pipeline se usa liblinear balanceado.Es un modelo rápido, interpretable y útil como línea base, pero su frontera es esencialmente lineal en el espacio transformado de entrada.

3.5.4 Random Forest

Random Forest fue visto por primera vez en una propuesta de Breiman en 2001 como una evolución del bagging con aleatorización adicional en la selección de variables [18]. El método construye B árboles sobre muestras bootstrap y, en cada partición, fuerza la búsqueda del mejor corte solo sobre un subconjunto aleatorio de variables mediante una regla de voto mayoritario:

$$\hat{\mathrm{y}}(\mathrm{x})=\mathrm{mode}\{\mathrm{h_b}(\mathrm{x})\}_{\mathrm{b}=1}^{\mathrm{B}}$$

O, en términos probabilísticos:

$$\hat{P}(y=c \mid x)=\frac{1}{B}\sum_{b=1}^{B}\mathbb{I}(h_b(x)=c)$$

La combinación de árboles poco correlacionados puede reducir varianza y mejorar generalización. En tráfico IoT es útil porque captura interacciones no lineales entre tiempos, banderas y ventanas sin normas específicas.

3.5.5 AdaBoost

AdaBoost surge en los trabajos de Freund y Schapire a mediados de los noventa como un éxito del boosting [19]. Su idea central es entrenar clasificadores débiles, aumentando el peso de las observaciones mal clasificadas anteriores. El clasificador final puede escribirse como:

 https://doi.org/10.5281/zenodo.20125818

$$F_T(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$$

El peso del clasificador t depende de su error ε_t:

$$\alpha_t = \frac{1}{2}\log\left(\frac{1-\varepsilon_t}{\varepsilon_t}\right)$$

Y los pesos de las observaciones se van actualizando como:

$$w_i^{(t+1)} \propto w_i^{(t)} \exp\left(\alpha_t \mathbb{I}\left(y_i \neq h_t(x_i)\right)\right)$$

AdaBoost es un método de boosting que combina muchos clasificadores débiles para construir un clasificador fuerte entrenando modelos de forma secuencial aumentando la importancia de lo mal clasificando por iteración llegando a una suma media de clasificador.

3.5.6 K vecinos más cercanos (KNN)

El clasificador k-Nearest Neighbors tiene su primera aparición en el trabajo de Cover y Hart de 1967 [20]. A diferencia de los modelos paramétricos, KNN clasifica cada observación a partir de sus k vecinos más cercanos bajo una métrica de distancia (normalmente euclídea):

$$N_k(x) = \arg\min_{S:\,|S|=k} \sum_{x_j \in S} d(x, x_j)$$

Su regla de predicción es:

$$\hat{y}(x) = \text{mode}\{\, y_j : x_j \in \mathcal{N}_k(x)\}$$

Es un método teóricamente simple y útil como referencia no paramétrica, pero muy sensible a la escala de las variables, a la densidad local y al ruido.

3.5.7 Árbol de decisión

Los árboles de decisión actuales se consolidan en CART por Breiman, Friedman, Olshen y Stone en 1984 [21], aunque la tradición de árboles de partición también incluye desarrollos como ID3 y C4.5. Trata de dividir recursivamente el espacio de entrada de la variable y el umbral que más reducen la impureza del nodo con índice de Gini:

$$G(t) = 1 - \sum_{c \in Y} p(\, c \mid t\,)^2$$

Y con entropía:

$$H(t) = -\sum_{c \in Y} p(\, c \mid t\,) \log p(\, c \mid t\,)$$

 https://doi.org/10.5281/zenodo.20125818

El mejor corte maximiza la reducción de impureza entre los nodos padre e hijos. Su mayor ventaja es la interpretabilidad, pero su principal debilidad, es la alta varianza y la tendencia al sobreajuste al no podarla o regularizarla.

3.5.8 Gaussian Naive Bayes

Los clasificadores Naive Bayes provienen del teorema de Bayes y la variante gaussiana, utilizada en aprendizaje automático para variables continuas, modela cada característica como normal [22]:

$$\mathrm{P}(\mathrm{y}=\mathrm{c}\mid \mathrm{x}) \propto \pi_{\mathrm{c}} \prod_{j=1}^{p} \mathcal{N}\left(x_j; \mu_{cj}, \sigma_{cj}^2\right)$$

Donde π_c es la probabilidad a priori de la clase. Aunque el modelo no funcione bien por no regirsse como el tráfico, sigue siendo útil por su rapidez, robustez y baratez computacional.

3.5.9 Análisis discriminante lineal (LDA)

El trabajo de Fisher en 1936 asentó la base del (LDA) [23] que asume que las clases siguen distribuciones gaussianas con medias diferentes pero una misma matriz de covarianza Σ reduciendo la decisión a una función lineal:

$$\delta_{\mathrm{c}}(\mathrm{x}) = \mathrm{x}^{\top}\Sigma^{-1}\mu_{\mathrm{c}} - \frac{1}{2}\mu_{\mathrm{c}}^{\top}\Sigma^{-1}\mu_{\mathrm{c}} + \log \pi_{\mathrm{c}}$$

Y la predicción adopta la forma:

$$\hat{y}(x) = \arg\max_{c} \delta_c(x)$$

Este modelo conecta estadística, reducción de dimensionalidad supervisada y clasificación. En este trabajo actúa como comparación lineal más estructurada.

3.5.10 Gradient Boosting

El Gradient Boosting Machine fue formulado por Friedman en 2001 como un descenso por gradiente en el espacio de funciones [16]. La construcción es aditiva:

$$\mathrm{F_M(x)} = \mathrm{F_{M-1}(x)} + \nu\gamma_{\mathrm{M}}\mathrm{h_M(x)}$$

Donde h_M aproxima el gradiente negativo de la pérdida en la iteración M, γ_M es el tamaño de paso y ν la tasa de aprendizaje. Trata de corregir secuencialmente el error de los árboles anteriores. En comparación con AdaBoost, es más general porque no se restringe a una única pérdida exponencial.

3.5.11 Support Vector Machine (SVM)

Support Vector Machine se consolidó como un clasificador de margen máximo desde el trabajo de Cortes y Vapnik sobre Support-Vector Networks [24]. Se centra en construir un hiperplano de decisión que separe las clases maximizando el margen entre los ejemplos más cercanos de cada clase, denominados vectores soporte. En su versión lineal separable, el problema puede formularse como:

$$\min_{w,b} \frac{1}{2}\|w\|^2 \quad \text{sujeto a} \quad y_i(w^\top x_i + b) \geq 1,\ i = 1, \ldots, n$$

Cuando los datos no se separan suavemente, se introducen variables de holgura ξ_i y un parámetro de penalización C, con un margen de:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \xi_i \quad \text{sujeto a} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i,\ \xi_i \geq 0$$

Tomando una decisión según:

$$\hat{y}(x) = \text{sign}(w^\top x + b) \ \text{ó sin calcular la tranformación: } \hat{y}(x) = \text{sign}\left(\sum_{i=1}^{n} \alpha_i y_i K(x_i, x) + b\right)$$

SVM es de utilidad cuando las clases presentan fronteras complejas pero definidas, aunque en datasets grandes o variados, es costoso computacionalmente, sobre todo con probabilidades calibradas, métricas de error o validación cruzada amplia.

3.6 Métricas de evaluación de los modelos e interpretabilidad XAI

Las métricas de aprendizaje supervisado más comunes se expresan como:

$$\text{Accuracy} = \frac{1}{n}\sum_{i=1}^{n} \mathbb{I}(\hat{y}_i = y_i), \quad \text{Precisión}_c = \frac{TP_c}{TP_c + FP_c}, \quad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c}$$

$$F1_c = \frac{2\,\text{Precision}_c\,\text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}, \quad \text{AUC} = \int_0^1 \text{TPR}(u)\ d\,\text{FPR}(u)$$

Donde:

$$\text{TPR} = \frac{TP}{TP + FN}, \qquad \text{FPR} = \frac{FP}{FP + TN}$$

En el caso multiclase, el ROC-AUC se computa bajo un esquema one-vs-rest donde se compara una curva de cada clase o su promedio macro.

La importancia por permutación se expresa como la caída de rendimiento al desordenar la característica j:

$$I_j = s(f, X, y) - s\big(f, \pi_j(X), y\big)$$

Donde $s(\cdot)$es una métrica de score y $\pi_j$permuta la columna j [15]. Shap y Eli5 tienen formas parecidas y prácticas de mostrar pesos o importancias de forma legible, aunque eli5 lo hace de forma más explicable.

La explicación DE Eli5n se representa mediante una función lineal simple:

 https://doi.org/10.5281/zenodo.20125818

$$g(x) = w_0 + \sum_{j=1}^{p} w_j x_j$$

ELI5 suele comunicar los coeficientes w_j o rankings de variables globales sobre los atributos que más empujan la predicción.

Para la explicación aditiva global y local, se usa SHAP, para una observación x y una característica j:

$$\phi_j(x) = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!\,(M - |S| - 1)!}{M!} \left[f_{S \cup \{j\}}(x) - f_S(x) \right]$$

Donde F es el conjunto de características y $M = | F |$ [13]. SHAP reparte la predicción entre variables mediante sumas, cuantifica cuánto aporta cada una a una observación concreta y permite agregaciones coherentes a nivel global.

Por último, LIME aproxima localmente al modelo complejo mediante:

$$\xi(x) = \arg\min_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g)$$

Equilibrando fidelidad local y simplicidad del modelo sustituto [14]. A diferencia de SHAP, LIME no busca una descomposición exacta de la predicción bajo axiomas aditivos, sino una aproximación interpretable.

ELI5 se usa como lectura global rápida y compacta; SHAP es una descomposición aditiva más rica, consistente y utilizable tanto local como globalmente; y LIME como explicación de vecindad, y soamente de manera local.

4. Resultados

4.1 Ánalisis exploratorio EDA

4.1.1 Ánalisis inicial completo

Después del preprocesado inicial, se procedió al análisis exploratorio. Se muestra una asimetría grave entre grupos: el tráfico dos_ddos domina la muestra original con el 76.849% del total, mientras que bruteforce apenas representa el 0.031 %. La Figura 3 refleja esa desproporción.

Para evitar un sesgo por una única familia de ataques, se construye una segunda versión mitigada del dataset, reteniendo una muestra de 7,075 observaciones dos_ddos, de acorde a la mediana porcentual de los demás.

 https://doi.org/10.5281/zenodo.20125818

No se utilizaron técnicas de remuestreo o métodos de generación sintética, como SMOTE, con el fin de mantener la distribución y estructura original del dataset y evitar que las fases posteriores incorporasen observaciones artificiales que pudieran alterar su interpretacion.

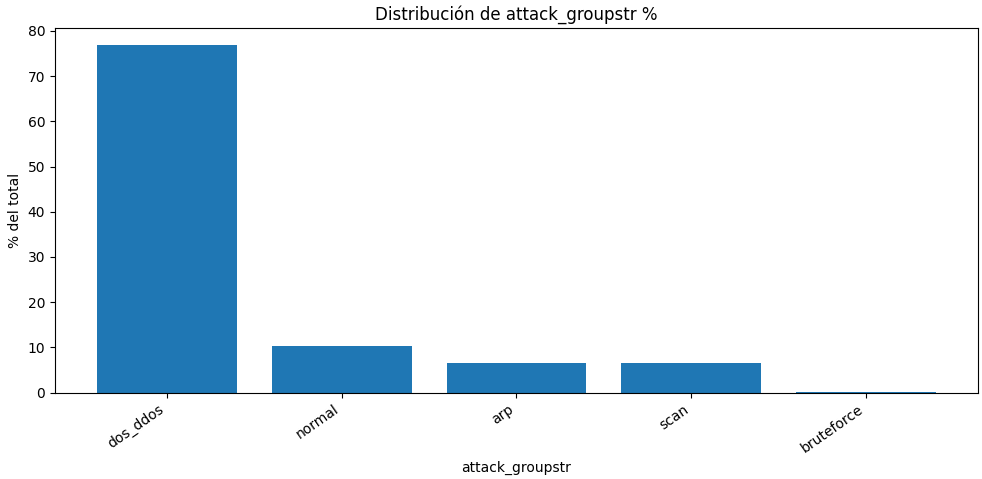


Figura 3. Distribución porcentual de los grupos de ataque antes de la mitigación.

La fase EDA no se agota en el reparto de clases, se tiene en cuenta el flujo de los diferentes paquetes en varios intervalos de tiempo, pero no se sabe cuánto se ha mandado unidireccionalmente, bidireccionalmente o no se ha mandado. Para ello, crearemos con los cuartiles de los paquetes totales, bidireccionalidad, interpretando hasta el tercer cuartil como unidireccionalidad y lo que se sobreponga a eso será bidireccional. Para las cuales haya columnas que especifiquen realmente el flujo de un lado a otro unidireccionalmente, simplemente se interpolarán a una nueva columna llamada flujo, que será la suma de las direcciones unidireccionales de cliente a servidor y servidor a cliente.

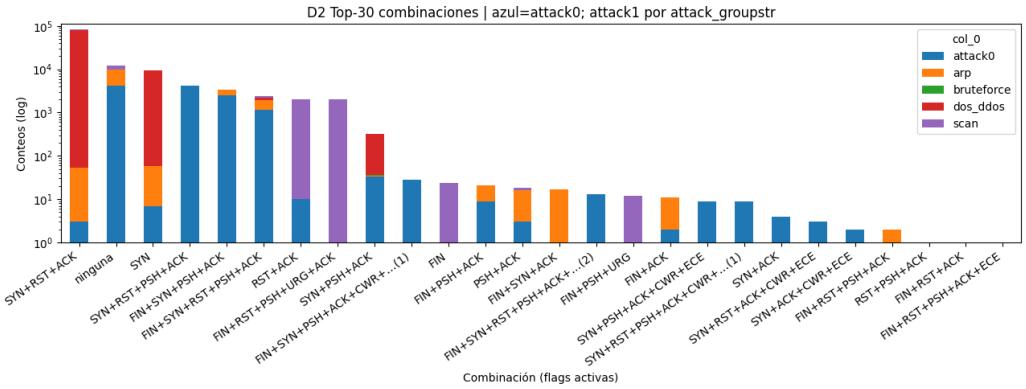


Figura 4. Análisis exploratorio de combinaciones de banderas y grupos de tráfico.

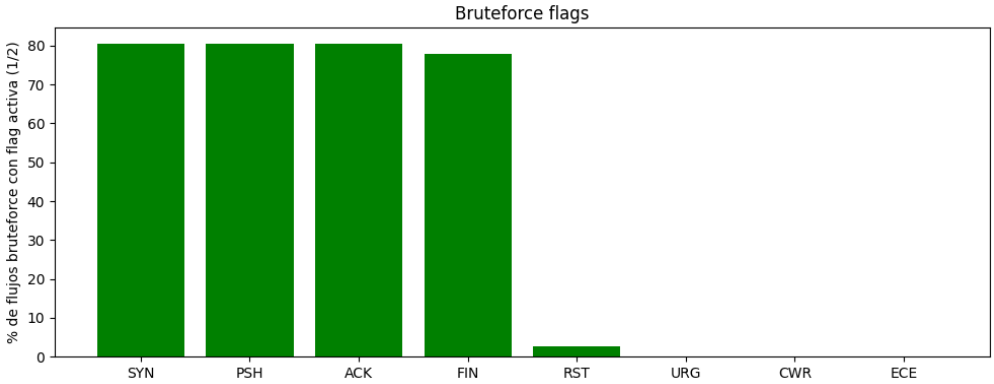


Figura 5. Distribución porcentual de tipos de flags en brute force SSH.

La Figura 4 sintetiza una relación exploratoria importante entre combinaciones de banderas y familias de tráfico. Se observó que las combinaciones dominadas por escaneo aparecen recurrentemente asociadas a flags de cierre (FIN) y, en algunos casos, URG, las combinaciones que incluyen ECE/CWR están mayoritariamente dominadas por tráfico benigno (attack0), consistente con señales de congestión/ECN más que con comportamiento malicioso.

Los ataques dos/ddos se concentran en combinaciones donde aparecen SYN/ACK y RST, lo que encaja con flujos muy cortos o reinicios frecuentes.

Los ataques dns-arp sobre todo con PSH y, en menor medida, con RST/FIN/SYN, aunque estos últimos son tcp y por ello debe interpretarse como una simple aparición sin relación ya que ARP no es TCP, usa DNS que es UDP.

Por último, en la Figura 5, se aprecia el análisis de los ataques por fuerza bruta (bruteforce), que aparece de forma repentina y en muy pocas combinaciones. Pero las que aparece depende de los paquetes SYN, PSH, ACK, y FIN, además de una baja concentración de RST, lo que puede significar que estos ataques no se cortan repentinamente, osea que se sabe perfectamente el objetivo y puerto abierto desprotegido, y continúan incluso logrando entrar al target.

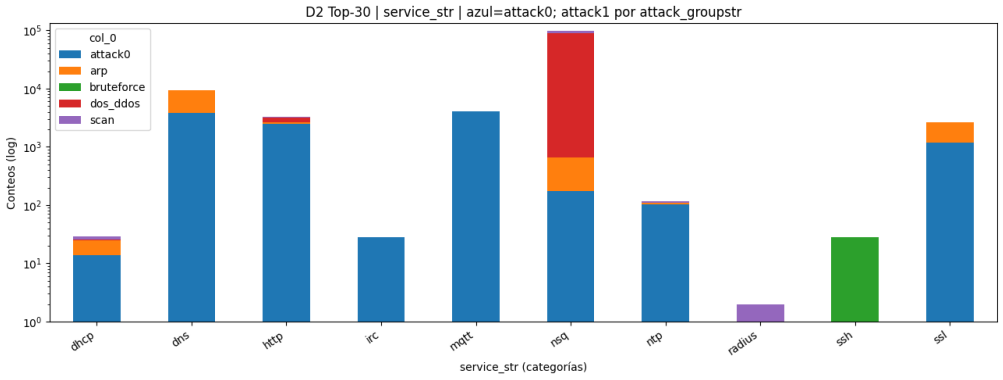


Figura 6. Proporción de clases respecto a su servicio.

Según la Figura 6, podemos concluir que se añade la variable de servicio y protocolo a la lista de atajos que eliminar. Los ataques de bruteforce se hacen todos con protocolo

SSH, y por tanto, habría que poner especial atención a estos. Los escaneos de red, sobre todo con RADIUS, que sirve para escaneos de puertos UDP 1812 y 1813 para identificar servidores vulnerables, y normalmente pueden realizarse con 2 paquetes. Además, un ataque DDoS puede durar hasta cinco veces más que el flujo de tráfico corriente y estos están dominados por una gran cantidad de datos.

4.1.2 Conclusión del análisis, coorelación, mitigación y depuración final

Se prepararon para el futuro dos datasets de análisis: uno con una proporción de dos_ddos ajustada y proporcional, no considerada sesgo al provenir de una muestra total ya sesgada por un entorno controlado, y otro sin ningún ajuste innecesario.

Se eliminaron variables de contexto que podían actuar como atajos, como puertos de origen y destino, protocolo y servicio, transformaciones por cuartiles, duplicados, colineales o dummies indirectos directamente dependientes unas de otras, todos los targets o sus derivadas, como attack_type y sus one-hot, y todas las correlaciones mayores al 95 % en valor absoluto con el target. Además, se revisó que las variables restantes no tuvieran relación directa con lo eliminado anteriormente y fueran aptas para el análisis y el ML posterior: duración, tasas, volumen, asimetría, flags, payload, IAT, active/inactive y ventana. Al final, se conservaron 51 de 123 columnas, es decir, el 41 % del dataset original

Se fijó random_state = 42 para garantizar reproducibilidad, y se usó para el filtrado de una proporción adecuada de los ataques dos_ddos, buscando que se pareciera más a otros grupos cuya proporción es del 6.5% sobre el valor central del dataset original. En el nuevo dataset, ddos representa aproximadamente el 24.5 %, quedándonos con una muestra del 7.82 % de los ataques ddos originales.

Por último, aunque se exploró la correlación de Pearson y Spearman entre variables, no se identificaron relaciones especialmente útiles o interpretables para el problema planteado que no fueran resolubles o interpretables de la misma manera, o que ya se hubieran identificado con la misma precisión en otros apartados.

4.2 Selección de k y estructura latente

Tras estandarizar y antes de fijar el número de grupos, se evalúa si antes se puede reducir el espacio y tiempo de cálculo y dimensionalidad. La Figura 7 muestra la varianza acumulada explicada por PCA de 51 variables operativas: aproximadamente 10 componentes retienen en torno al 80% de la información, 15 alrededor del 90 %, 20 cerca del 96% y 25 componentes rozan el 98 %.

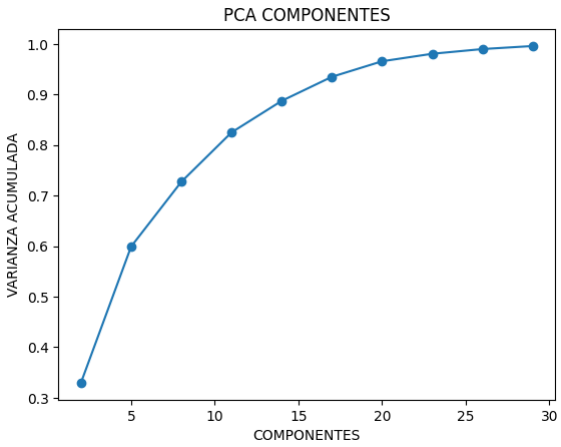


Figura 7. Varianza acumulada de los componentes principales en la fase de reducción dimensional.

Reducir la dimensionalidad antes de agrupar no solo alivia ruido y redundancia, sino que también hace más legible la geometría del espacio de flujos y evita que variables muy correlacionadas distorsionen las distancias, aunque solo retenga una proporción de la información original.

Uno de los tests realizados en los datasets reducidos y originados, mitigados y no mitigados, es el de la inercia que resume la compactación interna de los clusters a medida que aumenta k.

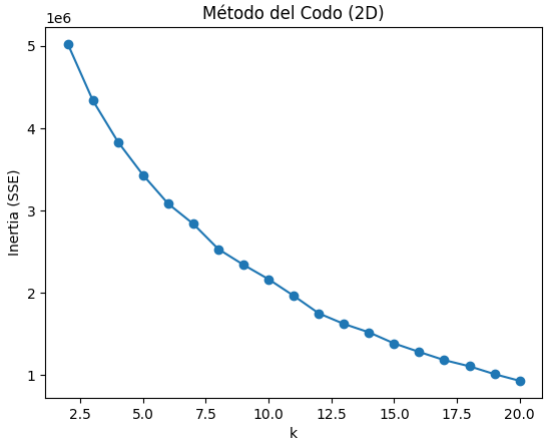


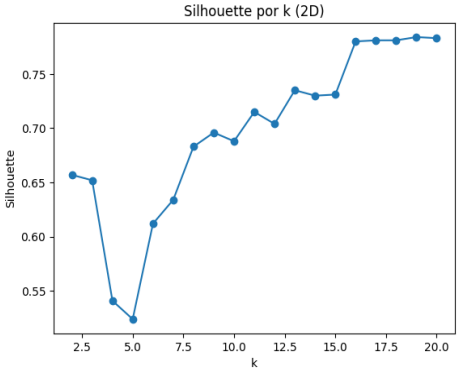
Figura 8. Curva de inercia para la selección de k clusters.

La Figura 8 muestra que no aparece un codo nítido o apreciable, y aunque aumente la compactación, no aporta la información necesaria para escoger un número de clusters específico.

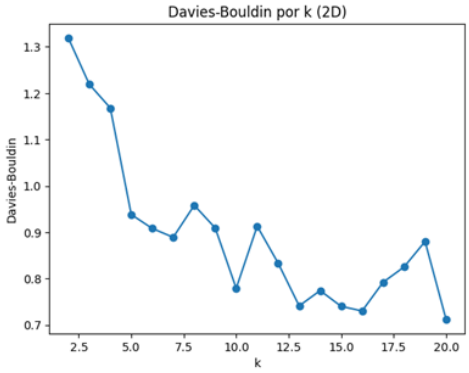
En la Figura 9 se estandarizan los otros tres tests de selección de clusters. Los coeficientes de Silhouette, que mide simultáneamente cohesión interna y separación respecto a grupos vecinos; cuanto mayor es, mejor estructurados están los clusters.

El factor Davies–Bouldin resume la similitud entre grupos a partir de su dispersión; cuanto menor es, mejor separación existe entre ellos.

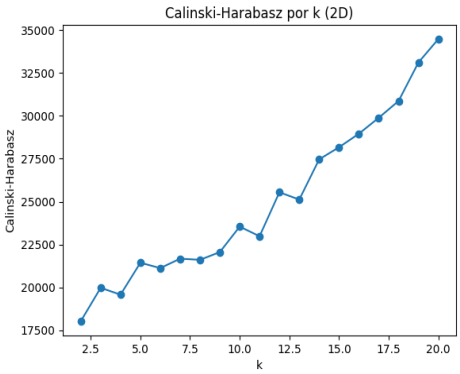
La constante Calinski–Harabasz compara dispersión entre clusters frente a dispersión interna de estos; cuanto mayor es, más clara es la estructura global.



(a) Silhouette



(b) Davies–Bouldin



(c) Calinski–Harabasz

Figura 9. Métricas de apoyo para la selección de clusters: Silhouette, Davies–Bouldin y Calinski–Harabasz.

Los criterios métricos tendieron a favorecer valores altos de K, especialmente en el dataset mitigado escalado, donde se observaron rangos de K=19–20, frente a K=17–20 en PCA. En el dataset completo, los rangos métricos fueron algo menores: K=16–17 en PCA y K=16–18 en datos escalados. No obstante, la lectura gráfica apuntó a soluciones más compactas e interpretables, con K=12 en PCA mitigado, K=10 en datos escalados mitigados y K=12 para el dataset completo.

En lo completo, se concluye con que k = 17 alcanza un valor de Silhouette de 0.781, un Davies–Bouldin de 0.792 y un Calinski–Harabasz de 29,888.345, que son los óptimos respecto a los demás valores en conjunto. Aunque la inercia no marca un codo evidente, Davies–Bouldin introduce algo de duda entre valores próximos y 17 clusters puede ser un motivo de sobrefragmentación, pero se adopta como compromiso entre compactación, separación e interpretabilidad respecto al conjunto de datos escalados y reducidos. Otras opciones incluyen k = 12, que permanece como solución visualmente muy alineada con las doce clases originales, pero menos sólida desde una perspectiva métrica.

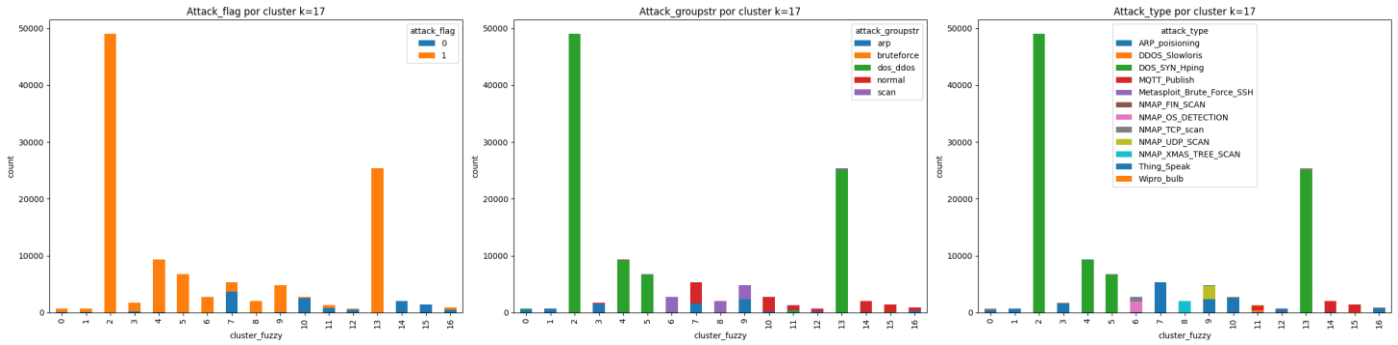


Figura 10. Distribución de targets binarios, agrupados y de 12 clases de clusters FCM con k = 17. 667

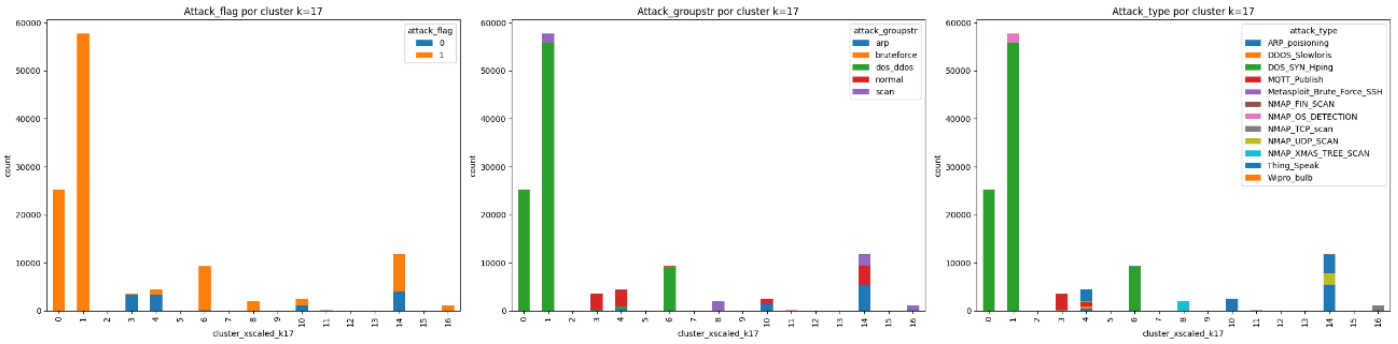


Figura 11. Distribución de targets binarios, agrupados y de 12 clases de clusters KM con k = 17. 668

Las Figuras 10 y 11 deben leerse de forma conjunta. La solución de clusters difusos fuzzy con k = 17 conserva mejor las transiciones y pertenencias parciales, por lo que ofrece una base interpretativa más útil, separada y fronteriza. En cambio, los clusters K-Means produce particiones más rígidas, compactas y nítidas, pero parece ser afectada a mayor nivel por la sobrefragmentación. En una prueba con otros k, no mejora el problema de la sobrefragmentación, y es tan grave como que para una k<7 sigue afectando de forma evidente. 669 670 671 672 673 674 675

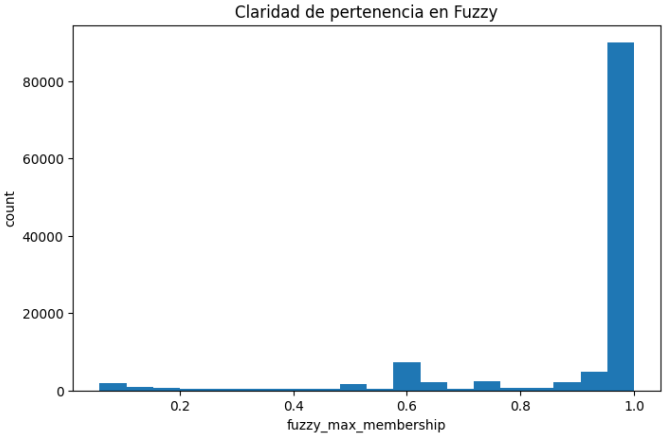


Figura 12. Gráfico del factor de pertenencia y nitidez. 676

En ambos casos, los clusters clasifican proporcionalmente las clases, aunque sí cartografían regiones densas, zonas periféricas y áreas mixtas: el grupo dos_ddos aparece particularmente distinguible, mientras que algunas clases raras y parte del tráfico benigno mantienen proximidades más difusas, aunque son estas las que tienen una mayor 677 678 679 680

pertenencia y nitidez con un nivel fpc=0.74 y 0.82 indicado en la Figura 12. Esta comparación anticipa las posibles clases que se separarán casi por completo en la fase supervisada y cuáles requerirán otra interpretación más cuidadosa en matrices de confusión, métricas de error y explicaciones locales.

4.3 Modelos de aprendizaje automático

4.3.1 Comparación inicial de modelos supervisados

La primera acción por tomar fue separar en un train-test de 75-25 el dataset a entrenar conformando el test a 29481 observaciones. La comparación entre algoritmos sitúa a Extra Trees y Random Forest entre los algoritmos de más alto del rendimiento medio cuando se usa el dataset completo sin mitigación, con XGBoost también hay un comportamiento muy competitivo, aunque más sensible a outliers en algunos folds, y es en el dataset completo cuando XGBoost tiene la mejor puntuación. Las Figuras 13 y 14 separan boxplots respecto a la estabilidad en ROC-AUC y en exactitud (accuracy).

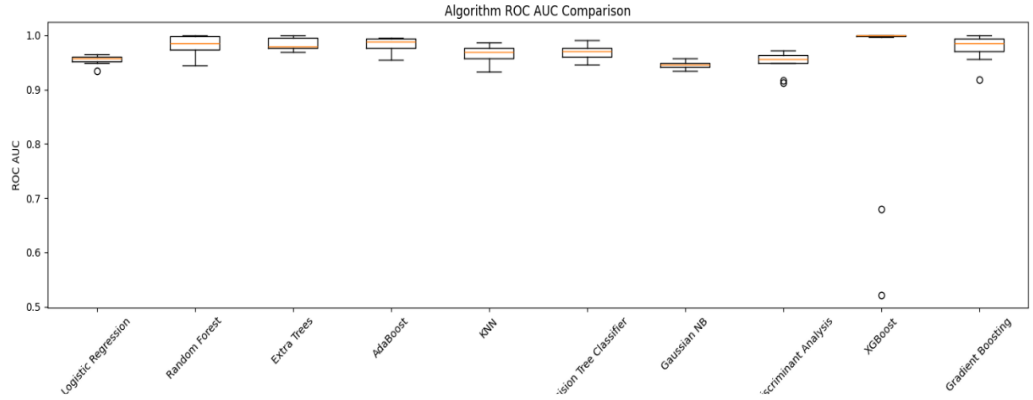


Figura 13. Boxplot del ROC-AUC en validación cruzada para los algoritmos supervisados comparados.

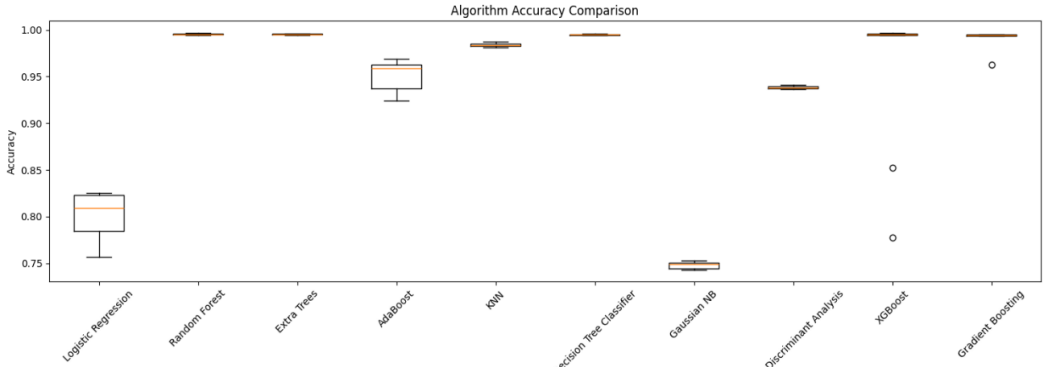


Figura 14. Boxplot de la exactitud en validación cruzada para los algoritmos supervisados comparados.

En el análisis preliminar con todo dos_ddos, Extra Trees alcanza una exactitud media del 99.534 % y un ROC-AUC medio del 98.341 %; después, el ajuste de hiperparámetros confirma la fortaleza de los dos candidatos principales. XGBoost obtiene como mejor

configuración learning_rate = 0.1, max_depth = 7 y n_estimators = 200, con un mejor score de 0.999064; por su parte, Extra Trees converge en max_depth = 10, max_features = log2 y n_estimators = 100, con un score de 0.996902.

Esto también nos dice que la duración del tráfico, agrupación, y tipo de paquetes que lo conforman son suficientes para determinar casi por completo el tipo y por ende si es o no un ataque y cual exactamente sin ver protocolos o el contenido de estos, y que no hace falta ningún método complicado de probabilidades o árboles, boosting o neighbours, una simple asignación de coeficiente es más que suficiente.

4.3.2 Evaluación final sobre el target de 12 clases

El bloque final de evaluación se apoya en el target original de 12 clases por ser el más informativo. El Word de partida insiste en que el objetivo no es solo separar tráfico normal frente a malicioso, sino comprobar hasta qué punto la misma tubería distingue subtipos finos de tráfico con significado operativo. La Figura 12 resume el rendimiento one-vs-rest del modelo final, que alcanza un ROC-AUC multiclase de 0.999.

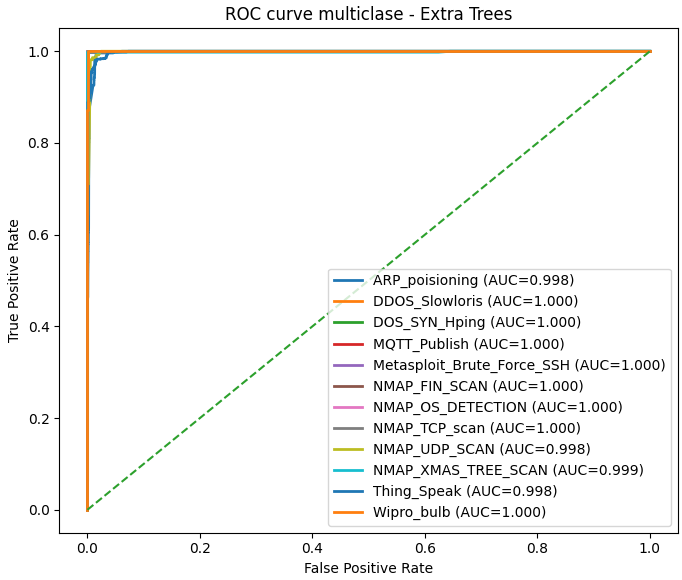


Figura 15. Curvas ROC multiclase one-vs-rest del modelo final Extra Trees sobre el target de 12 clases.

La Figura representa la separabilidad de cada clase frente al resto. Lo que se observa es una capacidad discriminativa muy alta en casi todas las categorías, coherente con un problema donde varias familias poseen firmas de flujo muy marcadas. Su casi perfecta curva puede ser indicativo de un patrón evidente entre las clases y el target que aún se conserva, aunque para otras clases con menor representación es más difícil.

Para ver realmente que no se interpreta bien por el modelo, se tiene que mirar la matriz de confusión, con una exactitud global del 98%, la Figura 13 presenta precisamente esa matriz de confusión del modelo final multiclase.

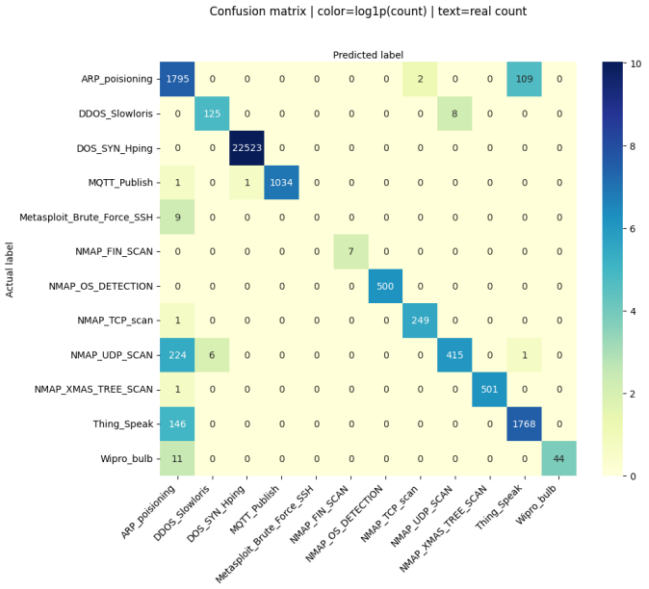


Figura 16. Curvas ROC multiclase one-vs-rest de Extra Trees sobre el target de 12 clases.

La matriz representa qué clases se predijeron de forma casi perfecta y en cuáles persistieron zonas de solape. A primera vista, se observó una gran facilidad para reconocer patrones muy marcados, como DOS_SYN_Hping o MQTT_Publish, mientras que ARP_poisoning concentró parte de sus errores hacia NMAP_UDP_SCAN. Esta confusión puede explicarse por sus semejanzas estructurales, ya que ambas clases comparten tramas cortas, repetitivas y con escasa respuesta de retorno. En cambio, las clases volumétricas y mejor definidas fueron distinguidas con mucha claridad. También destacó el caso de Metasploit_Brute_Force_SSH, cuyo soporte extremadamente bajo (9 casos) impidió una identificación fiable.

La comparación entre targets confirmó que, al reducir el problema a clasificación binaria o a 5 grupos, gran parte de esta ambigüedad desapareció, por tanto, la dificultad principal no estuvo en detectar anomalías, sino en tipificar con precisión las clases finas de tráfico.

Los resultados globales mostraron un rendimiento muy alto en los tres niveles de target. En el caso binario, el modelo separó con gran precisión el tráfico benigno y no benigno, con valores cercanos a 1.00 y muy pocos errores. En el target de 5 grupos también se mantuvo un comportamiento sólido, con precisión aproximada de 0.97 y F1 igual o superior a 0.97 en casi todos los grupos. Finalmente, en el target original de 12 clases, el modelo alcanzó una exactitud del 98 %, un F1 ponderado de 0.98 y un ROC-AUC multiclase de 0.999, lo que confirmó una gran capacidad discriminativa de carácter general.

4.4 Interpretabilidad XAI global y local

La fase XAI confirma que el sistema no decide a partir de una única variable aislada, sino a partir de combinaciones de duración, cabeceras, ventanas, banderas TCP, SYN, payload y ritmos bidireccionales. La Figura 17 muestra la jerarquía de importancia global del modelo final.

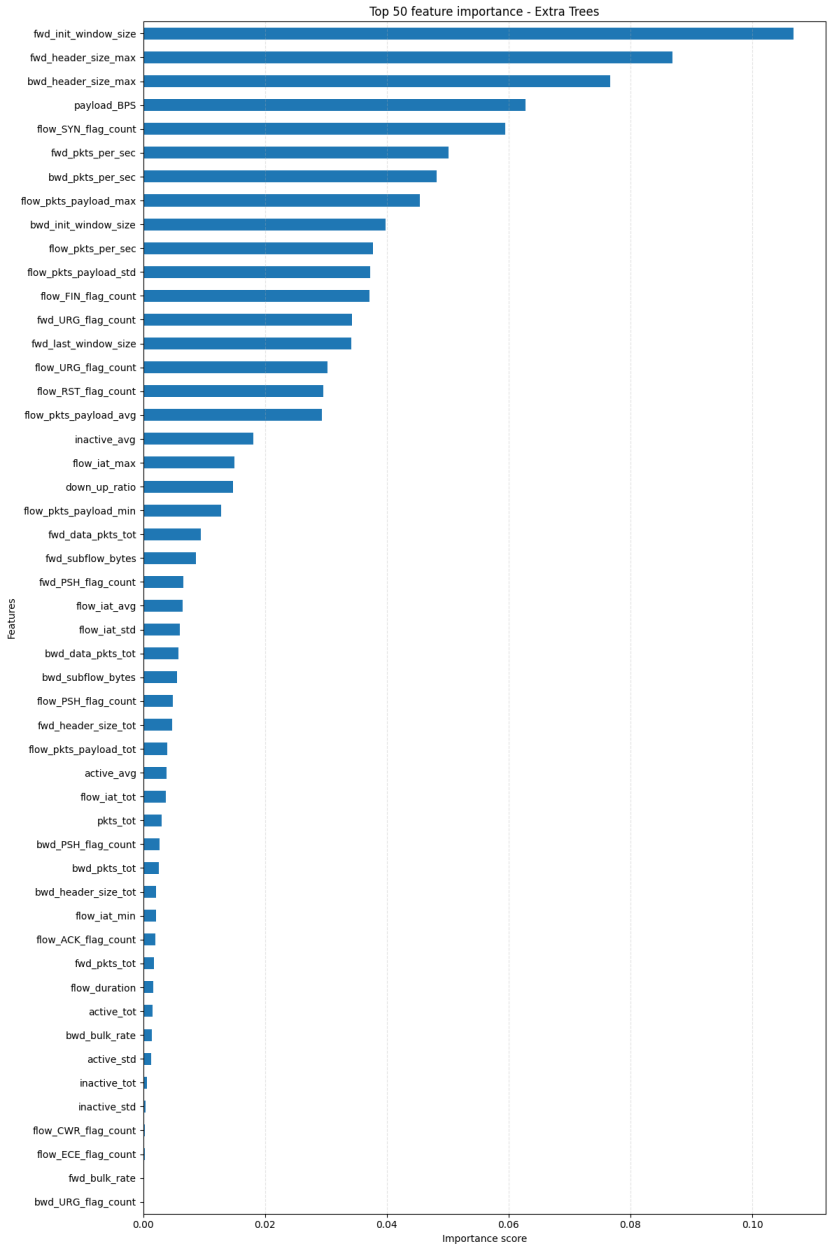


Figura 17. Importancia global de variables del modelo final multiclase.

La Figura 17 representa el peso agregado de las características a escala global. Lo que se observa es una mezcla de variables temporales, de carga útil, de ventana y de estructura de subflujos, destacando: fwd_last_window_size, flow_iat_min, fwd_subflow_bytes, payload_BPS, flow_pkts_payload_min y bwd_pkts_per_sec.

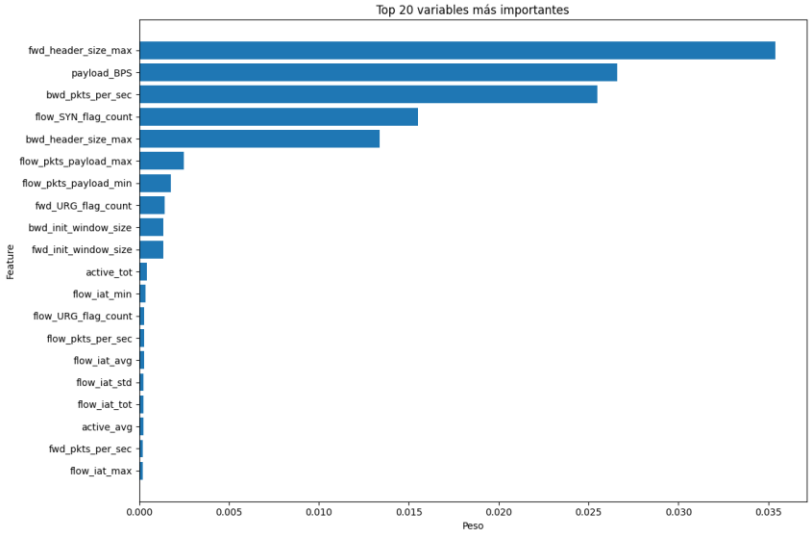
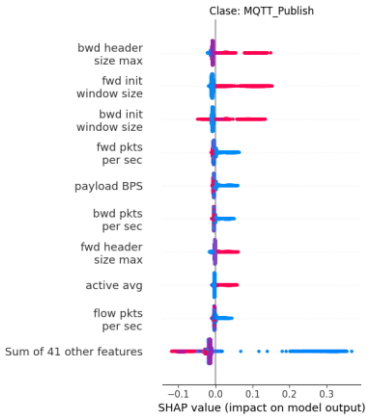
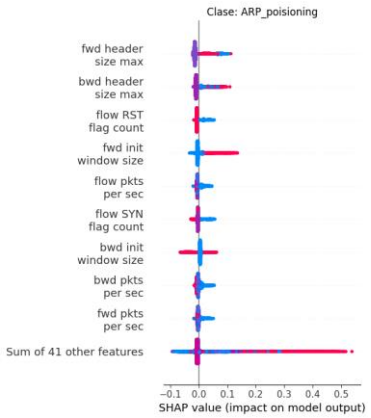


Figura 18. Importancia global de las 20 variables más influyentes según la lectura ELI5 del modelo final

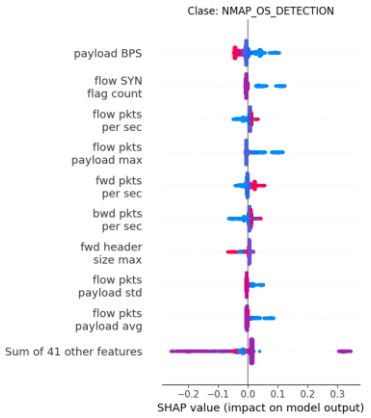
La Figura 18 ofrece una segunda lectura global, más compacta, de las mismas dependencias. La Figura 18 muestra que una agrupación reducida de variables acapara gran parte del peso del modelo, especialmente aquellas relacionadas con el tamaño de las cabeceras, la cantidad de carga útil transmitida, la frecuencia de paquetes por segundo y la presencia de determinados indicadores de conexión. Sugiriendo gran importancia de los rasgos estructurales del flujo: cuánto se transmite, con qué ritmo circulan los paquetes, cómo se organizan las cabeceras y qué marcas aparecen en la comunicación. A partir de ese primer bloque dominante, la importancia cae de forma clara, por lo que el resto de variables actúa más como apoyo secundario.



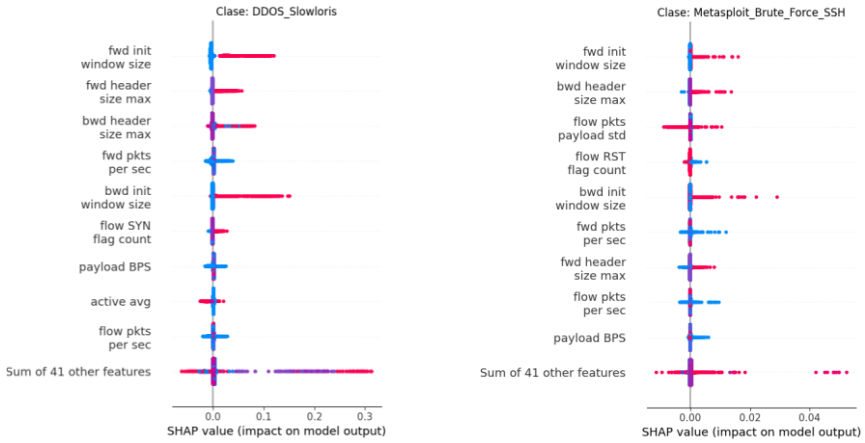
(a) Sensor de temperatura



(b) Envenenamiento arp



(c) Escaneos os



(e) Denegación de servicio (f) Fuerza bruta

Figura 19: Explicaciones SHAP por clase para una clase de tráfico normal, nmap scans, arp poisoning, dos_ddos y bruteforce.

La Figura 19 recoge explicaciones SHAP por clase para varios comportamientos representativos del tráfico analizado. Para interpretar la Figura, debe considerarse que el color rojo indica valores altos de cada variable respecto a su distribución, mientras que el azul representa valores bajos:

- En MQTT_Publish, se observan variables relacionadas con tamaños de cabecera, tamaños de ventana, paquetes por segundo, carga útil y actividad del flujo, con valores más moderados en variables de intensidad y patrones más estables en la estructura del flujo, coherentes para un sensor de temperatura, más continua y menos agresiva.

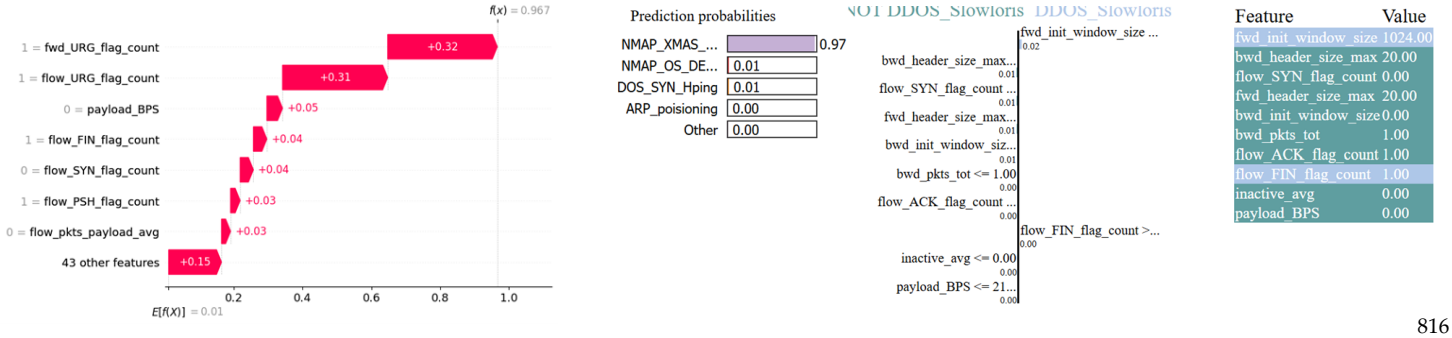
- En ARP_poisoning, el modelo se apoya sobre todo en valores de cabeceras, ventanas, flags y ritmo de paquetes con valores bajos de carga útil, tiempos reducidos entre paquetes y señales repetitivas que encaja perfectamente con tramas pequeñas, frecuentes y de escasa respuesta de retorno.

- Esto también ayuda a explicar su cercanía con otras clases ligeras como NMAP_UDP_SCAN y NMAP_OS_DETECTION, que destacan por valores altos en variables de carga útil por segundo, flags SYN y paquetes por segundo, junto con variaciones en la carga útil del flujo ya que tratan de provocar respuestas diagnósticas del host que hacia un volumen bruto elevado y sostenido.

- En DDOS_Slowloris, se notan valores altos en variables de tiempos de conexión, tamaños de ventana y comportamiento temporal del flujo, coherente con un ataque basado en mantener conexiones abiertas o prolongadas, más que en generar únicamente un volumen masivo de tráfico.

- En Metasploit_Brute_Force_SSH, la explicación es menos estable y debe interpretarse con cautela por su bajo soporte en test. Aun así, las variables

destapan un patrón basado en intentos sucesivos de acceso, con pausas variables, carga moderada y cambios en el ritmo del flujo, lo que encaja con un comportamiento de fuerza bruta más secuencial que volumétrico.



(a) Shap waterfall local

(b) Exploración Lime

Figura 20: Explicación local mediante LIME y SHAP para una observación aleatoria asociada a un escaneo tipo XMAS y su contribución por clases.

La Figura 20 ilustra el valor de LIME y SHAP como herramienta de explicación por caso. Representa en un caso específico qué variables empujan una predicción concreta hacia una clase de escaneo y cuáles la alejan de alternativas vecinas. SHAP ofrece una lectura más estructural y por clase, mientras LIME permite auditar instancias concretas, pesos y valores que empujan esa predicción.

En conjunto, la lectura cruzada de importancia global, ELI5, SHAP y LIME permite resumir cinco patrones relevantes:

- El tráfico normal se asocia a mayor regularidad, mayor separación temporal entre paquetes, payload moderado y respuesta bidireccional más tranquila. Thing_Speak y Wipro_bulb refuerzan este patrón con señales de ventana y cadencia relativamente corrientes.
- Los escaneos se definen con poco intercambio real de datos, búsqueda de respuesta del host y cabeceras con huellas específicas. NMAP_UDP_SCAN destaca por la escasa respuesta backward, NMAP_FIN_SCAN por la presencia de banderas FIN, como su propio nombre indica, NMAP_OS_DETECTION por variables de ventana y ritmo de respuesta, y NMAP_TCP_scan por cabeceras y estructura de sesión más que por volumen.
- DoS yDDoS comparten presión temporal sobre el flujo, pero DDOS_Slowloris se relaciona con conexiones abiertas e irregularidad, mientras DOS_SYN_Hping concentra envíos rápidos, volumétricos y paquetes SYN.
- ARP poisoning presenta un patrón de tramas pequeñas, repetitivas y muy frecuentes, favorecido por payload mínimo relativamente alto dentro del flujo y tiempos mínimos entre paquetes muy bajos.
- Brute force SSH muestra comportamiento más secuencial: muchos intentos consecutivos, pausas variables, repeticiones y menos volumen que un ataque volumétrico. Aunque se debe ser cauto por extremadamente reducido soporte que tiene.

5. Discusión

5.1 Aportación de los clústeres

Uno de los riesgos de incluir clustering en un escrito de aprendizaje automático es usarlo como una ilustración cosmética. Aquí no sucede eso. La fase no supervisada aporta tres elementos analíticos concretos: detecta que dos_ddos ocupa una región muy compacta del espacio, evidencia zonas mixtas entre ataques ligeros y tráfico benigno, y muestra que una parte minoritaria de las observaciones no debería forzarse a una única asignación rígida. Por ende, las membresías fuzzy son útiles no porque "expliquen" por sí solas el fenómeno, sino porque señalan dónde la frontera entre clases es estructuralmente ambigua.

Por un lado, valores como k = 12 o k=5 son visualmente atractivos porque recuerdan a las doce clases originales; por otro, k = 17 resulta más coherente al combinar inercia, Silhouette, Davies–Bouldin y Calinski–Harabasz. Esa discrepancia entre el "ganador gráfico" y el "ganador métrico" es instructiva ya que se debe elegir solo por parecido superficial con las etiquetas conocidas, sino por su capacidad matemática y geométrica para describir subregímenes, fragmentaciones y fronteras reales del espacio de flujos.

5.2 Implicaciones metodológicas y transferibilidad

Se sugiere que una selección cuidadosa de variables de flujo puede ofrecer un rendimiento muy elevado sin inspección profunda de paquete como primer filtro. Esto es metodológicamente relevante porque reduce dependencias respecto a payloads completos, puertos específicos, contenido cifrado o reglas específicas por servicio o protocolo. Y busca demostrar una capacidad discriminativa sustentada en patrones de flujo más transferibles y menos dependientes de un entorno concreto.

Sin embargo, la transferibilidad y escalabilidad debe entenderse de forma correcta. No significa que los mismos pesos, centroides o importancias se reproduzcan intactos en otra red. Significa que el flujo de trabajo //limpieza, reducción de atajos, mitigación del sesgo, clustering, clasificación y explicación// puede reutilizarse para ajustar un sistema al historial propio de una organización con intereses parecidos.

5.3 Limitaciones y amenazas a la validez

El trabajo tiene al menos seis limitaciones que conviene explicitar.

Primero, la procedencia del dataset: RT-IoT2022 es un entorno controlado y sesgado y, por tanto, separa comportamientos de forma probablemente más limpia que una red institucional con ruido operativo, coexistencia de aplicaciones y tráfico de fondo.

Segundo, el desequilibrio extremo de algunas clases, especialmente Metasploit_Brute_Force_SSH, DDOS_Slowloris y DOS_SYN_Hping , condiciona tanto la macromedia como la solidez de cualquier conclusión por clase.

Tercero, la fase XAI aquí es post hoc: hace legible la decisión del modelo, pero no convierte automáticamente al modelo en causal ni en universalmente robusto.

Cuarto, la propia mitigación de dos_ddos mejora la exploración, pero altera la distribución original y, por ello, debe interpretarse como una decisión analítica y no como fotografía fiel del laboratorio.

Quinto, la abundancia de flujos muy breves, con muchos ceros o casi ceros, puede favorecer separaciones especialmente limpias en ataques de sondeo o respuesta mínima, algo que no siempre se reproducirá igual fuera del laboratorio.

A estas limitaciones se añade una advertencia metodológica importante, si se reincorporaran atajos como puertos o servicios, el rendimiento probablemente subiría todavía más, pero el experimento dejaría de medir con la misma pureza la capacidad de los descriptores de flujo para generalizar a otros contextos. En consecuencia, los resultados deben leerse como evidencia de capacidad discriminativa controlada y de utilidad metodológica, no como validación externa definitiva.

6. Conclusiones y trabajo futuro

La integración sistemática del contenido del análisis permite además reforzar el carácter interpretativo del trabajo, especialmente en EDA, clustering, clasificación e XAI.

Desde el punto de vista empírico, el EDA confirmó que RT-IoT2022 está fuertemente condicionado por el diseño del laboratorio: el grupo de ataque de denegación de servicio domina la muestra, abundan flujos extremadamente cortos y varias firmas ofensivas se manifiestan mediante pocos paquetes, baja respuesta de retorno y patrones muy marcados de banderas. Esto justifica la mitigación del desbalanceo y la decisión de no depender exclusivamente de puertos, protocolo, servicio u otros atajos relacionados para caracterizar el tráfico.

La fase de agrupación no supervisada mostró que una composición de k = 17 ofrece un compromiso razonable entre separación, compactación y legibilidad aceptables y que Fuzzy C-Means aporta una lectura más útil que K-Means en observaciones fronterizas. La inercia no presentó un codo claro; los factores de Silhouette y Calinski–Harabasz favorecieron valores relativamente altos de k, mientras el coeficiente de Davies–Bouldin introdujo matices y dudas pero no invalidó la elección final. En conjunto, el clustering reveló que algunas regiones geométricas coinciden bien con clases conocidas, mientras otras reflejan ambigüedad estructural, poca nitidez y separación entre escaneos ligeros, ARP poisoning y tráfico benigno de baja intensidad.

La fase supervisada confirmó un rendimiento sobresaliente en los tres niveles de target. En target binario, el sistema separa tráfico normal y malicioso de forma prácticamente perfecta. En 5 grupos mantiene resultados muy altos salvo en bruteforce, afectado por la escasez de casos. En el escenario de 12 clases, el modelo final alcanza una exactitud del 98% y un ROC-AUC multiclase de área 0.999, con errores localizados principalmente en clases de soporte reducido o firmas parcialmente solapadas, como ARP_poisioning, NMAP_UDP_SCAN y Metasploit_Brute_Force_SSH.

La capa de interpretación XAI reforzó que la decisión del modelo descansa en combinaciones de variables temporales, volumétricas y de control de flujo, y no en una sola magnitud dominante o de etiqueta. Las importancias globales, ELI5, SHAP por clase y

LIME local convergen en una misma idea: el tráfico normal es más regular y espaciado, los escaneos dependen de cabeceras, banderas y respuestas parciales; los ataques dde denegación de servicio comparten presión sobre el flujo pero con temporalidades distintas y el envenenamiento arp y ataques por fuerza bruta exigen interpretaciones más finas por su estructura repetitiva o su soporte limitado.

Como proyecto futuro, se puede mejorar en al menos cinco direcciones: (i) ampliar todavía más las explicaciones locales de LIME y SHAP para clases concretas y errores relevantes, (ii) incorporar matrices de confusión adicionales para los targets binario y de 5 grupos, (iii) contrastar el pipeline con capturas fuera del laboratorio y con tráfico organizativo real anonimizado, (iv) estudiar más variantes de clustering y aprendizaje costesensible que traten mejor las clases raras sin depender de un recorte explícito del grupo dominante y (v) evaluar el impacto controlado de reintroducir variables contextuales como servicio o protocolo para comparar explicabilidad frente a rendimiento puro, (vi) reforzar la estabilidad, reproducibilidad y trazabilidad mediante más métodos de validación, optimización y seguimiento experimental con herramientas de validación cruzada con plataformas como Weights & Biases y MLflow, técnicas de autologging para registrar hiperparámetros, métricas y resultados, optimización bayesiana con Optuna y pipelines con ColumnTransformer para reducir el riesgo de data leakage en el pipeline, herramientas de autoprofiling, y paneles interactivos con Streamlit.

Referencias

1. Sharmila, B.S.; Nagapadma, R. Quantized Autoencoder (QAE) Intrusion Detection System for Anomaly Detection in Resource-Constrained IoT Devices Using RT-IoT2022 Dataset. *Cybersecurity* **2023**, *6*, 127–134, doi:10.1186/s42400-023-00178-5.

2. Yost, J.R. The Origin and Early History of the Computer Security Software Products Industry. *IEEE Annals of the History of Computing* **2015**, *37*, 46–58.

3. Manivannan, D. Recent Endeavors in Machine Learning-Powered Intrusion Detection Systems for the Internet of Things. *Journal of Network and Computer Applications* **2024**, *229*, doi:10.1016/J.JNCA.2024.103925.

4. Sharma, B.; Sharma, L.; Lal, C.; Roy, S. Explainable Artificial Intelligence for Intrusion Detection in IoT Networks: A Deep Learning Based Approach. *Expert Syst. Appl.* **2024**, *238*, 122–177, doi:10.1016/j.eswa.2023.121751.

5. Chen, X.; Liu, M.; Wang, Z.; Wang, Y. Explainable Deep Learning-Based Feature Selection and Intrusion Detection Method on the Internet of Things. *Sensors* **2024**, *24*, 1983, doi:10.3390/s24165223.

6. Marín Díaz, G. FAS-XAI: Fuzzy and Explainable AI for Interpretable Vetting of Kepler Exoplanet Candidates. *Mathematics* **2025**, *13*, doi:10.3390/math13233796.

7. Marín Díaz, G. A Unified Fuzzy–Explainable AI Framework (FAS-XAI) for Customer Service Value Prediction and Strategic Decision-Making. *AI* **2025**, *7*, 3, doi:10.3390/ai7010003.

8. Marín Díaz, G. Quality Management in Chemical Processes Through Fuzzy Analysis: A Fuzzy C-Means and Predictive Models Approach. *ChemEngineering* **2025**, *9*, doi:10.3390/chemengineering9030045.

9. Marín Díaz, G. Fuzzy C-Means and Explainable AI for Quantum Entanglement Classification and Noise Analysis. *Mathematics* **2025**, *13*, doi:10.3390/math13071056.

10. James C. Bezdek *Pattern Recognition with Fuzzy Objective Function Algorithms*; 1981;

11. Chen, T.; Guestrin, C. XGBoost: A Scalable Tree Boosting System. In Proceedings of the Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining; Association for Computing Machinery, August 13 2016; Vol. 13-17-August-2016, pp. 785–794.

12. Geurts, P.; Ernst, D.; Wehenkel, L. Extremely Randomized Trees. *Mach. Learn.* **2006**, *63*, 3–42, doi:10.1007/s10994-006-6226-1.

13. Lundberg, S.M.; Allen, P.G.; Lee, S.-I. *A Unified Approach to Interpreting Model Predictions*; 2017;

14. Ribeiro, M.T.; Singh, S.; Guestrin, C. "Why Should i Trust You?" Explaining the Predictions of Any Classifier. In Proceedings of the Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining; Association for Computing Machinery, August 13 2016; Vol. 13-17-August-2016, pp. 1135–1144.

15. Fisher, A.; Rudin, C.; Dominici, F. *All Models Are Wrong, but Many Are Useful: Learning a Variable's Importance by Studying an Entire Class of Prediction Models Simultaneously*; 2019; Vol. 20;.

16. Friedman, J.H. *999 REITZ LECTURE GREEDY FUNCTION APPROXIMATION: A GRADIENT BOOSTING MACHINE 1*; 2001; Vol. 29;.

17. Berkson, J. Application of the Logistic Function to Bio-Assay. *J. Am. Stat. Assoc.* **1944**, *39*, 357–365, doi:10.1080/01621459.1944.10500699.

18. Breiman Leo Random Forests. **2001**, 5–32.

19. Freund, Y.; Schapire, R.E. *Journal of Computer and System Sciences s SS1504 Journal of Computer and System Sciences*; 1997; Vol. 55;.

20. Cover, T.M.; Hart, P.E. *Approximate Formulas for the Information Transmitted Bv a Discrete Communication Channel*; 1952; Vol. 24;.

21. Leo Breiman; Jerome H. Friedman; Richard A. Olshen; Charles J. Stone *Classification and Regression Trees*; 1984;

22. John, G.H.; Langley, P. *Estimating Continuous Distributions in Bayesian Classiiers*; Morgan Kaufmann Publishers, 1995;

23. R. A. FISHER *THE USE O F MULTIPLE MEASUREMENTS IN TAXONOMIC PROBLEMS I. DISCRIMINANT BUNCTIONS*; 1936;

24. Cortes, C.; Vapnik, V.; Saitta, L. *Support-Vector Networks Editor*; Kluwer Academic Publishers, 1995; Vol. 20;.

25. B. S. and R. Nagapadma. "RT-IoT2022 ," UCI Machine Learning Repository, 2023. [Online]. Available: https://doi.org/10.24432/C5P338.